

DOM Cross-Site Scripting (XSS) via Insecure Web Messages

Summary

A DOM-based Cross-Site Scripting (DOM XSS) vulnerability was identified due to insecure handling of messages received through the browser's `postMessage()` API. The application processed untrusted message data without proper validation or sanitization, allowing arbitrary JavaScript execution in the victim's browser.

This issue was successfully demonstrated in the PortSwigger Web Security Academy lab environment.

Vulnerability Details

Vulnerability Type: DOM-Based Cross-Site Scripting (DOM XSS)

Attack Vector: Cross-window communication (`window.postMessage`)

Severity: High

OWASP Top 10: A03:2021 – Injection

CWE: CWE-79 – Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

Description

The application listens for incoming messages using the browser's `postMessage()` API. However, the received message is inserted into the DOM without validating the sender's origin or sanitizing the message content.

An attacker can host a malicious page containing an iframe that loads the vulnerable application and sends a crafted message. When processed by the target application, the malicious payload executes JavaScript in the context of the victim's session.

Proof of Concept (PoC)

```
<iframe src="https://LAB-ID.web-security-academy.net/"
onload="this.contentWindow.postMessage('<img src=1 onerror=print()>','*')">
</iframe>
```

Exploitation Steps

1. Load the vulnerable application inside an iframe.
2. Wait until the page finishes loading.
3. Send a malicious message using `window.postMessage()`.
4. The application receives the message.
5. The message is rendered into the DOM without sanitization.
6. JavaScript executes successfully, demonstrating DOM XSS.

Impact

Successful exploitation could allow an attacker to:

- Execute arbitrary JavaScript in a victim's browser.
- Steal session tokens (where applicable).
- Perform actions on behalf of authenticated users.
- Modify page content.
- Conduct phishing attacks.
- Redirect users to malicious websites.
- Potentially compromise user accounts depending on application functionality.

Root Cause

The application trusted data received through the `postMessage()` API without:

- Verifying the sender's origin.
- Validating message structure.
- Sanitizing HTML before rendering.

This allowed attacker-controlled input to become executable content inside the DOM.

Remediation

To mitigate this vulnerability:

- Validate the sender using `event.origin`.
- Accept messages only from trusted origins.
- Treat all incoming message data as untrusted.
- Sanitize user-controlled content before inserting it into the DOM.
- Prefer `textContent` instead of `innerHTML` whenever possible.
- Implement a strong Content Security Policy (CSP).
- Avoid using wildcard (`*`) origins unless absolutely necessary.

References

- OWASP Cross Site Scripting Prevention Cheat Sheet
- PortSwigger Web Security Academy
- MDN Web Docs – `window.postMessage()`

Proof

Evidence:

- Screenshot demonstrating successful payload execution (`print()` dialog).

